JUnit

Unit tests for Java

Juan López Rubio juan.lopez-rubio@upc.edu

06-10-2024

Contents

JUnit 101 **2**
Create a new project with Maven enabled . . . 2
Add the JUnit dependency . . . 3
Create a new Java class . . . 3
Create the Unit Tests . . . 5
Run the tests . . . 7
Coverage . . . 8
Best Practices . . . 10
Use Hamcrest . . . 11

Exercises **13**
Exercise 1 . . . 13
Questions . . . 13

JUnit 101

JUnit is a unit testing framework for Java. It is used to write and run repeatable tests. In this tutorial, we will learn how to use JUnit to test our Java code using IntelliJ IDEA.

Create a new project with Maven enabled

We will create a simple Java project with Maven enabled. This will allow us to manage dependencies (for example JUnit itself) and run tests easily.

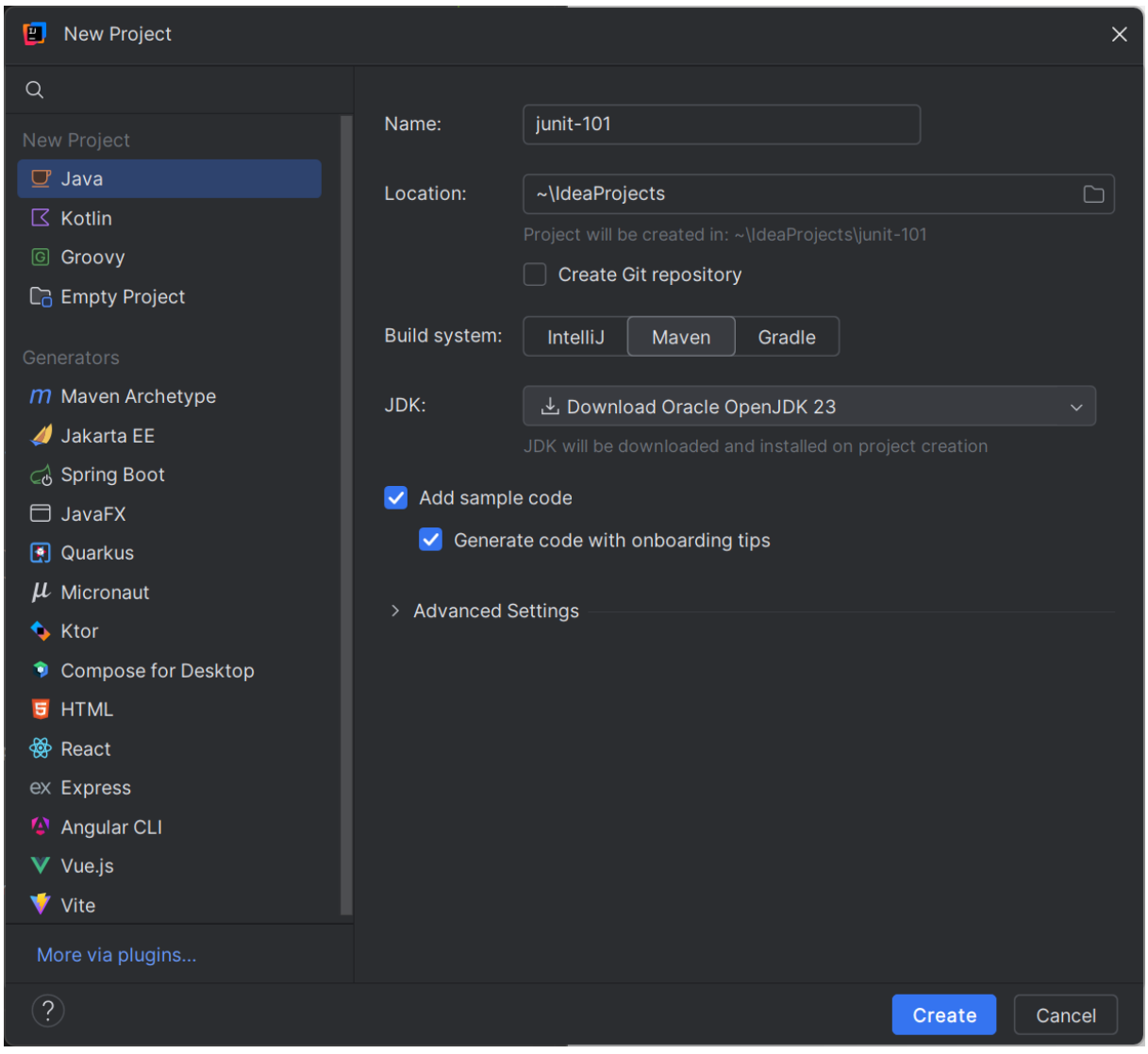


Figure 1: Create a new project

Add the JUnit dependency

We add the JUnit dependency to the `pom.xml` file. This file manages the project's dependencies. A dependency is a library that our project uses and that should be installed in the project's classpath. In this case we are installing JUnit 4.13.2.

```
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter-engine</artifactId>
        <version>5.9.1</version>
        <scope>test</scope>
    </dependency>
```

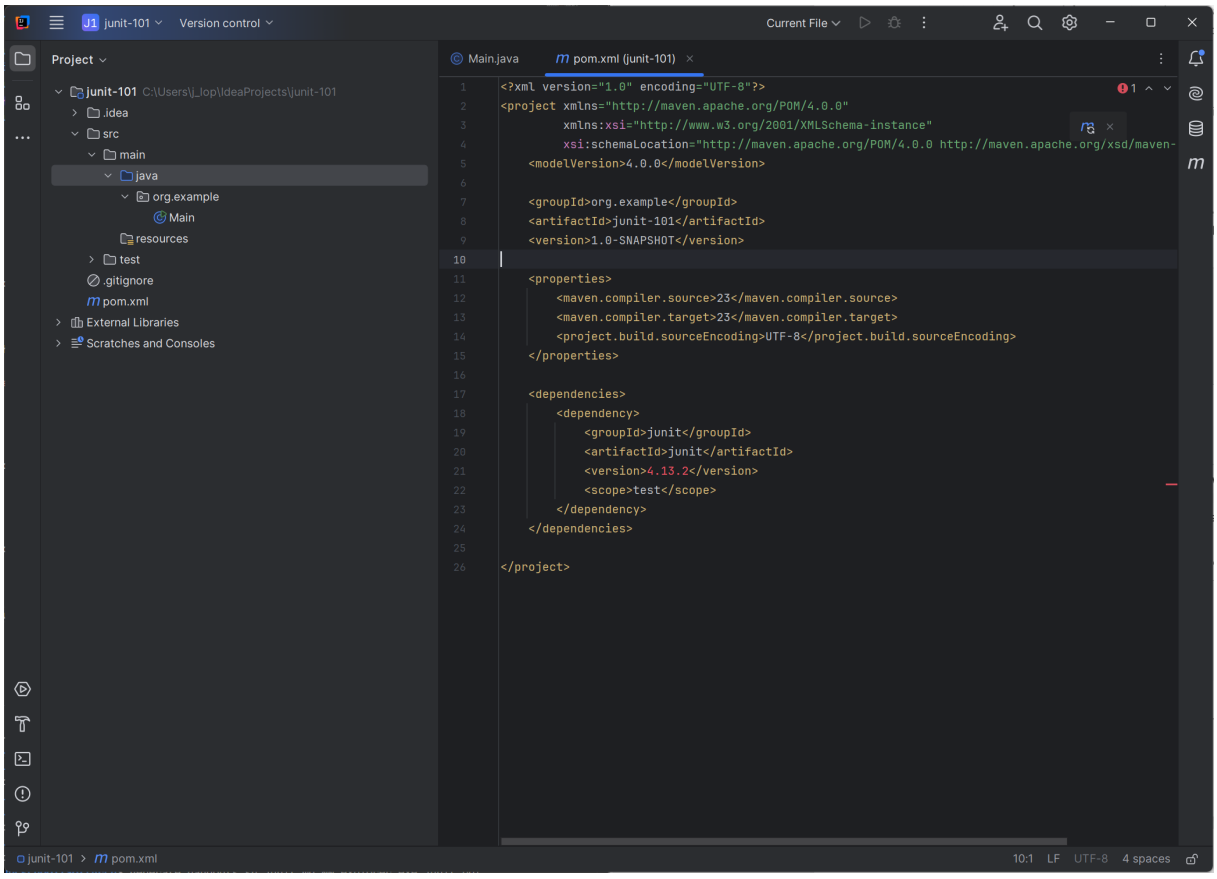


Figure 2: Add Maven dependency

Create a new Java class

We create a new Java class called Calculator. This class will have four methods: add, subtract, multiply, and divide. The add, subtract, and multiply methods will take two integers as arguments and

return their sum, difference, and product, respectively. The divide method will take two integers as arguments and return their quotient. If the second argument is zero, it will throw an IllegalArgumentException.

Please be sure to add this class in the `main` folder, and inside the `org.example` package.

```java
package org.example;

public class Calculator {

    public int add(int a, int b) {
        return a + b;
    }

    public int subtract(int a, int b) {
        return a - b;
    }

    public int multiply(int a, int b) {
        return a * b;
    }

    public int divide(int a, int b) throws IllegalArgumentException {
        if (b == 0) {
            throw new IllegalArgumentException("Division by zero is not
               allowed.");
        }
        return a / b;
    }

    public static void main(String[] args) {
        Calculator calculator = new Calculator();
        System.out.println("5 + 3 = " + calculator.add(5, 3));
        System.out.println("5 - 3 = " + calculator.subtract(5, 3));
        System.out.println("5 * 3 = " + calculator.multiply(5, 3));
        System.out.println("5 / 3 = " + calculator.divide(5, 3));
    }
}
```

We can run the main method to test the Calculator class and see its output.

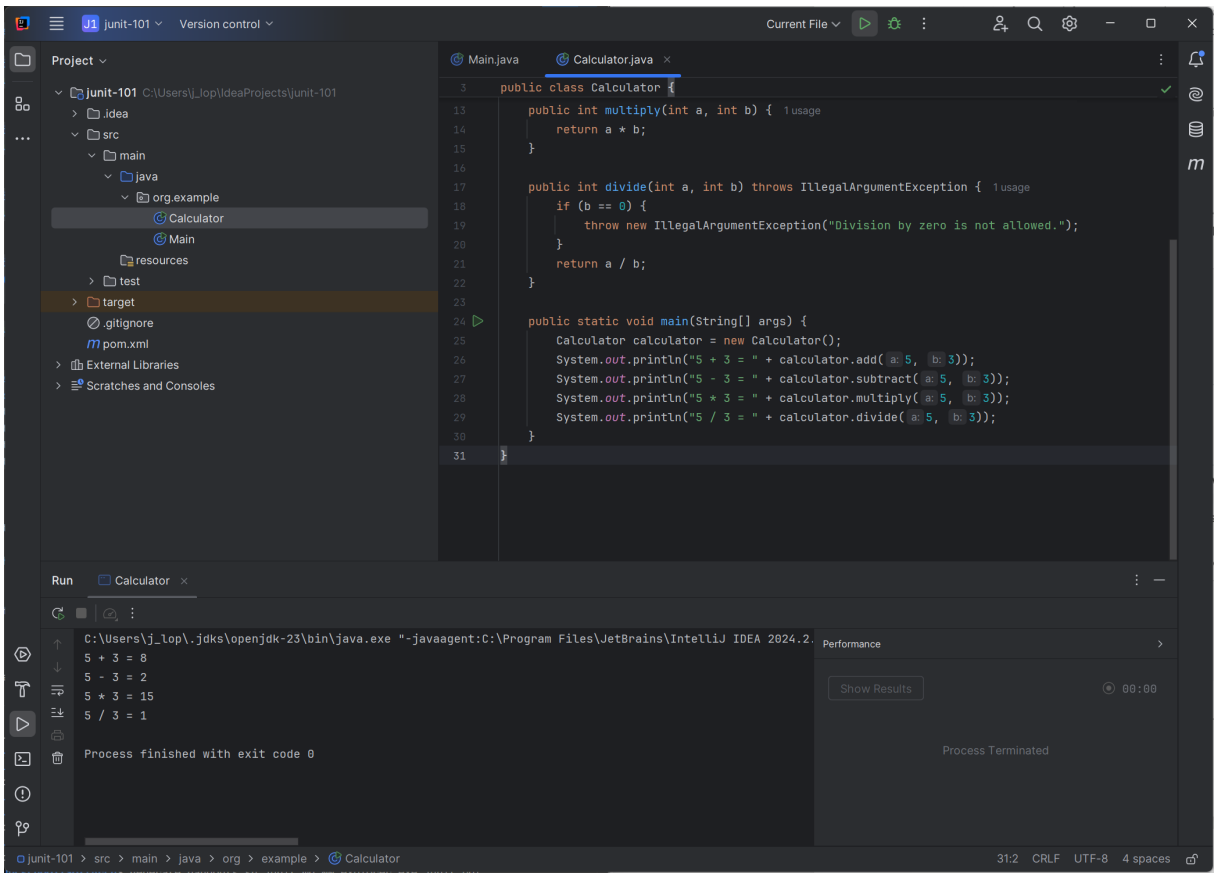


Figure 3: Create the main code

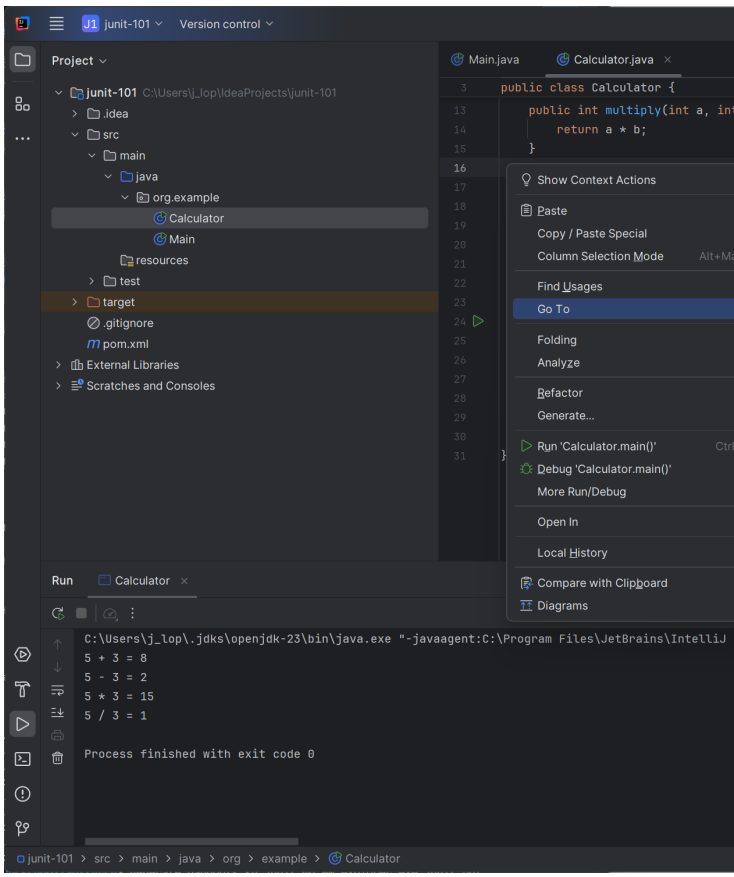
Create the Unit Tests

Now we will create a new Java class called `CalculatorTest`. This class will contain the unit tests for the Calculator class. We will use JUnit annotations to define the test methods.

This class must be created in the `test` folder and it should be in the same package as the Calculator class.

One way to simplify this, is to use the IntelliJ IDEA IDE. We can create a test class by right-clicking on the class we want to test and selecting `'Go to'-> 'Test'` or by pressing `Ctrl`+`Shift`+`T`.

A new dialog will appear, where we can select the methods we want to test. A new empty class will be created with the selected methods. In addition to this, we can choose to create `setUp` and `tearDown` methods if we want to run some code before and after each test. This methods are useful



to initialize and clean up resources required by the tests.

One example of the test class is the following:

```java
package org.example;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class CalculatorTest {
    Calculator calculator =new Calculator();
    @Test
    public void add(){
        assertEquals(8,calculator.add(5, 3));
    }
    @Test
    public void subtract(){
        assertEquals(2,calculator.subtract(5,3));
    }
    @Test
    public void multiply(){
        assertEquals(15, calculator.multiply(5,3));
    }
    @Test
    public void divide(){
```

```
        assertEquals(1,calculator.divide(5,3));
    }
    @Test
    public void divideByZero() {
        assertThrows(IllegalArgumentException.class, ()->calculator.divide
            (5,0));
    }
}
```

The `@Test` annotation is used to mark a method as a test method. On the other hand, the `assertEquals` method is used to check that the expected value is equal to the actual value. If it's not, the test fails. Obviouslly, we can use other methods like `assertTrue`, `assertFalse`, `assertNull`, `assertNotNull`, `assertSame`, `assertNotSame`, `assertArrayEquals`, etc.

Notice that the methods does not need to be called `add`, `subtract`, etc. We can use any name we want, obviously is recommended that they identify the operation that is being tested or the condition that is being checked. The important thing is that the method is annotated with `@Test`.

Also notice that the `divideByZero` method is testing that an `IllegalArgumentException` is thrown when dividing by zero. Now, the `division` operation is checked in two differents tests: `divide` and `divideByZero`. This is a good practice, because we are testing the normal behavior of the method and the exceptional behavior. You can add as many tests as you want to check the different conditions of a method.

It's possible that IntelliJ doesn't recognize the JUnit annotations. We can fix this by pressing the `Load Maven Changes` button in the top right corner of the editor. This will reload the project and download the missing dependencies. Other option is to right-click on the project folder and select `'Maven '-> 'Reload Project'`.

Run the tests

While we are in the `CalculatorTest` class, we can run the tests by pressing the green arrow next to the class name. This will run all the tests in the class and show the results in the `Run` tab. Now, all will be in in green, meaning that all the tests passed.

We can also run specific tests by right-clicking on the test method and selecting `'Run'-> 'CalculatorTest.add()'`, for example.

Assert methods check that the expected value is equal to the actual value. If it's not, the test fails. For example if we change the testAdd method to:

```
  assertEquals(10, calculator.add(5, 3));
```

The test will fail and the output will be:

```
java.lang.AssertionError:
Expected :10
Actual   :8
```

Coverage

IntelliJ IDEA also provides a code coverage tool. This tool shows which lines of code are executed during the tests. We can run it by right-clicking on the project folder and selecting `'Run'-> 'Run with Coverage'`. Depending on the version of IntelliJ IDEA, this option can be in `'Run'-> 'More Run/Debug'-> 'Run with Coverage'`.

This will run all the tests and show the results in the `Coverage` tab. This tab shows the % of classes, methods, lines and branches covered by the tests. Obvioully, the goal is to have 100% coverage, but this is not always possible or necessary.

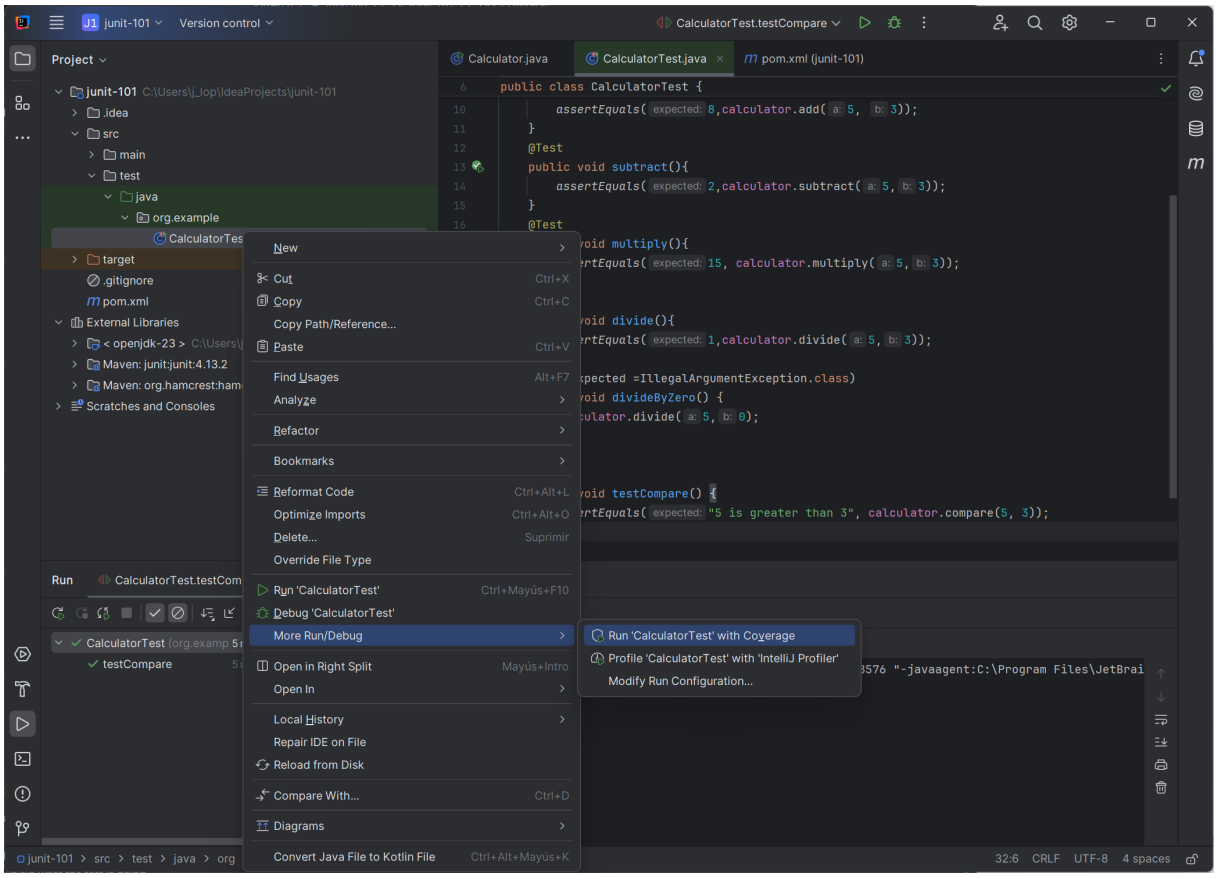


Figure 4: Run test with coverage

Let's add a new operation that has several branches.

```
public String compare(int a, int b) {
    if (a > b) {
        return a + " is greater than " + b;
    } else if (a < b) {
        return a + " is less than " + b;
    } else {
        return a + " is equal to " + b;
    }
}
```

Supose that we add the following test:

```
@Test
public void testCompare() {
    assertEquals("5 is greater than 3", calculator.compare(5, 3));
}
```

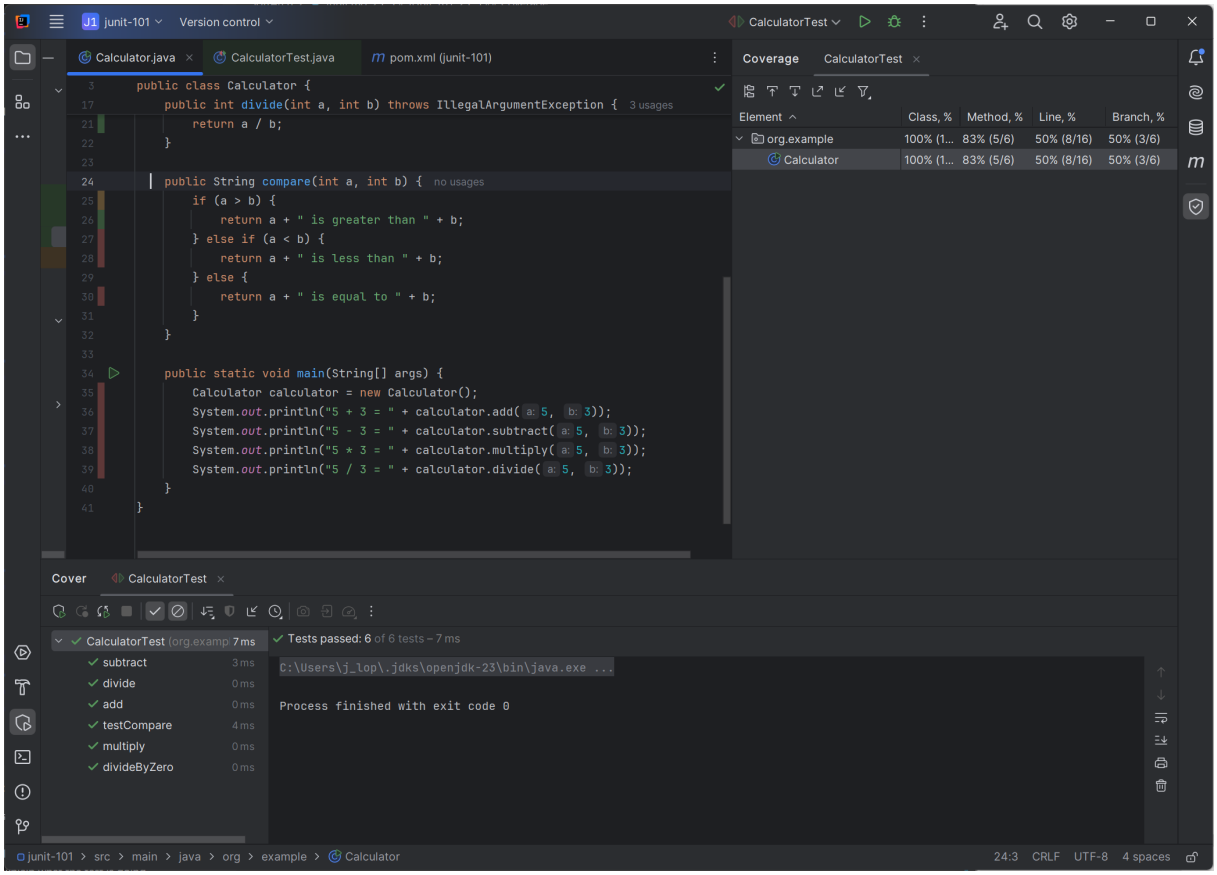


Figure 5: Coverage results

In our case, we see that have 83% method coverage (we are not testing `Calculator.main()`), 50%

line coverage and 50% branch coverage.

If we go to the source code, we will see that some lines are highlighted in green, and others in red. These are the lines that are covered and not covered by the tests, respectively. This is useful to check if we are testing all the code paths.

Our new code only are tested in 1 of the 3 branches. We should add a new test to cover the `else if` and `else` branch.

```
@Test
public void testCompareEqual() {
    assertEquals("5 is equal to 5", calculator.compare(5, 5));
}

@Test
public void testCompareLess() {
    assertEquals("3 is less than 5", calculator.compare(3, 5));
}
```

Now if we run the coverage tool again, we will see that all the branches are covered.

Best Practices

Imagine that we have a `Book` class with three fields: title, author, and year.

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class BookTest {

    @Test
    public void testBookEquality() {
        Book b1 = new Book("1984", "George Orwell", 1949);
        Book b2 = new Book("1984", "George Orwell", 1949);

        // This is ok
        assertEquals(b1, b1);

        // This is not ok
        assertEquals(b1, b2);
    }
}
```

The second assertion will fail because the `equals` method is used to compare two objects for equality. By default, the `equals` method in the `Object` class compares the memory addresses of the objects, not their contents. To fix this, we have several options:

Redefine equals method

We can override the `equals` method in the `Book` class to compare the fields of the objects. This is the recommended approach when we want to compare objects based on their contents. But, take into account that we should also override the `hashCode` method, because if two objects are equal, they should have the same hash code. These methods are used by some collections to check for duplicates.

```
public class Book {
    ...

    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }
        if (obj == null || getClass() != obj.getClass()) {
            return false;
        }
        Book book = (Book) obj;
        return year == book.year && title.equals(book.title) && author.
           equals(book.author);
    }

    @Override
    public int hashCode() {
        return Objects.hash(title, author, year);
    }
}
```

Now equality is based on the title, author, and year of the book and not on the memory address of the `Book` object. We can test this by using the `assertEquals` method.

Use Hamcrest

Another option is to use the Hamcrest library, which provides a set of matchers that can be used to compare objects based on their fields.

To use Hamcrest, we need to add the dependency to the `pom.xml` file:

```
    <dependency>
        <groupId>org.hamcrest</groupId>
        <artifactId>hamcrest</artifactId>
        <version>3.0</version>
    </dependency>
    <dependency>
        <groupId>org.hamcrest</groupId>
        <artifactId>hamcrest-library</artifactId>
        <version>3.0</version>
```

```
        <scope>test</scope>
    </dependency>
```

Now we can use the `samePropertyValuesAs` matcher to compare the fields of two objects.

```
import org.junit.Test;
import static org.hamcrest.MatcherAssert.assertThat;
import static org.hamcrest.Matchers.*;

public class BookTest {

    @Test
    public void testBookEquality() {
        Book b1 = new Book("1984", "George Orwell", 1949);
        Book b2 = new Book("1984", "George Orwell", 1949);

        // Using Hamcrest to compare fields
        assertThat(b1, samePropertyValuesAs(b2));
    }
}
```

This will also compare the fields of the `Book` objects and not their memory addresses, without the need to override the `equals` method.

Exercises

Exercise 1

Let's implement a managing system for a library. We will have a `Library` class that will have a list of books. Each book will have a title, an author, and a year of publication. The `Library` class will have the following methods:

- `addBook(Book book)`: adds a book to the library.
- `removeBook(Book book)`: removes a book from the library.
- `getBooksByAuthor(String author)`: returns a list of books by a given author.
- `getBooksByYear(int year)`: returns a list of books published in a given year.

Create the `Book` class and the `Library` class. You should also implement a `Main`class with a `main` method that creates a `Library` object, and provides a command line interface to interact with the library. The user should be able to add, remove, and search for books by author or year. Finally, write the unit tests for this system.

Questions

1. Is easy to test the `Main` class? Why? For the moment, you can avoid testing the `Main` class.